

Compresión y Restauración de Imágenes usando Análisis de Componentes Principales (PCA) Paralelo

Estefanny Sanluis Carcaño
Khristopher Kurt Morales Noyola
Instituto Politécnico Nacional (IPN)
Ingeniería en Inteligencia Artificial
Materia: Cómputo Paralelo

Resumen

El Análisis de Componentes Principales (PCA) es una técnica muy popular en el aprendizaje no supervisado que nos ayuda a reducir la dimensionalidad de los datos[cite: 1]. El problema real surge cuando queremos aplicarla en imágenes de alta resolución, ya que las operaciones matemáticas consumen muchísimos recursos computacionales[cite: 1]. Por ello, en este trabajo proponemos una implementación en paralelo orientada a acelerar tanto la compresión como la restauración de imágenes a color (RGB) usando PCA[cite: 1]. A diferencia de las versiones secuenciales clásicas, que tardan demasiado debido a su complejidad matemática de $\mathcal{O}(N^3)$, nosotros aprovechamos el compilador Numba para distribuir todo este trabajo entre los distintos núcleos del procesador[cite: 1]. Adicionalmente, abordamos el problema de la evaluación del peso de los archivos, justificando el uso de formatos crudos para medir la compresión matemática real. Con este proyecto buscamos comprobar empíricamente cómo el paralelismo reduce drásticamente los tiempos de ejecución sin afectar la calidad visual.

Palabras clave: Cómputo Paralelo, Análisis de Componentes Principales, Machine Learning, Compresión de Imágenes, Numba[cite: 1].

1. Introducción

En las materias de aprendizaje automático nos enseñan que el Análisis de Componentes Principales (PCA) juega un papel clave para extraer las características más importantes de un set de datos[cite: 1]. Es una herramienta matemática excelente, pero cuando la llevamos a la práctica en el terreno del procesamiento de imágenes (para comprimirlas y restaurarlas), el costo computacional se dispara de una manera brutal[cite: 1].

Esto ocurre porque el algoritmo nos exige calcular una matriz de covarianza y luego proyectar matrices gigantes de píxeles[cite: 1]. Si a esto le sumamos que una imagen a color (RGB) requiere que repitamos todas estas matemáticas pesadas tres veces (una por cada canal: rojo, verde y azul), un procesador que trabaja de forma secuencial rápidamente se convierte en un cuello de botella[cite: 1].

Para solucionar y entender este problema a fondo, en este proyecto nos propusimos desarrollar desde cero una versión de PCA enfocada totalmente en paralelizar estas multiplicaciones matriciales[cite: 1]. Construimos dos versiones en código Python: una secuencial "tradicional" y otra versión paralela acelerada mediante el compilador de Numba[cite: 1].

2. El Reto que Enfrentamos

El principal reto al que nos enfrentamos al programar este proyecto no fue solo entender las matemáticas del PCA, sino lidiar con las limitaciones físicas de la computadora y la forma en la que se maneja la memoria.

Cuando abrimos una imagen para procesarla, Python no ve un "dibujo", ve una matriz numérica gigantesca. El algoritmo PCA requiere multiplicar esta matriz por su transpuesta. En términos de ciencias de la computación, esto tiene una complejidad de $\mathcal{O}(N^3)$ [cite: 1]. Esto significa que si nuestra imagen crece un poco en resolución, la cantidad de multiplicaciones que el procesador tiene que hacer crece de forma cúbica.

Durante nuestras primeras pruebas con código secuencial (usando simples ciclos `for`), el programa tardaba muchísimo tiempo en procesar incluso imágenes de tamaño mediano, y en ocasiones, la memoria RAM llegaba a su límite. El reto consistió en rediseñar la forma en la que la computadora lee estos ciclos, obligándola a dividir el esfuerzo matemático entre todos sus núcleos disponibles al mismo tiempo, esquivando los bloqueos nativos de Python.

3. Marco Teórico: Las Matemáticas del PCA

Pensemos en nuestra imagen como una matriz gigante X , donde las dimensiones son $N \times D$ (las filas N representan los píxeles y las columnas D sus características)[cite: 1].

3.1. Centrado de los Datos

El primer paso es calcular el promedio (la media μ) de cada columna y restárselo a nuestra matriz original para que todo quede centrado en el origen (cero)[cite: 1]:

$$\mu_j = \frac{1}{N} \sum_{i=1}^N X_{ij} \quad (1)$$

$$X_c = X - \mu \quad (2)$$

3.2. Cálculo de la Covarianza y los Autovectores

Aquí calculamos una matriz de covarianza Σ que nos dice cómo se relacionan los píxeles entre sí. Para obtenerla, multiplicamos la matriz transpuesta por sí misma[cite: 1]:

$$\Sigma = \frac{1}{N-1} X_c^T X_c \quad (3)$$

Una vez obtenida, resolvemos el problema para encontrar los autovectores V y los autovalores λ [cite: 1]:

$$\Sigma V = \lambda V \quad (4)$$

Como nuestro objetivo es la compresión, no los guardamos todos; elegimos solo los K autovectores más fuertes. A esta nueva matriz la llamaremos V_K [cite: 1].

3.3. Compresión y Restauración

Proyectamos nuestra matriz centrada X_c multiplicándola contra nuestros componentes principales V_K , obteniendo una matriz comprimida Z que pesa muchísimo menos que la original[cite: 1]:

$$Z = X_c V_K \quad (5)$$

Para la restauración, multiplicamos Z por la transpuesta de V_K y le sumamos la media original[cite: 1]:

$$X_{restaurada} = Z V_K^T + \mu \quad (6)$$

4. Metodología e Implementación

Para llevar esto a cabo, diseñamos un flujo de trabajo que se divide en tres grandes etapas: la paralelización del núcleo matemático, la gestión del color y, finalmente, la correcta selección del formato de imagen para evaluar el peso.

4.1. Paralelización de la Multiplicación de Matrices

Sabiendo que el principal cuello de botella es la multiplicación de matrices densas[cite: 1], decidimos no usar funciones prefabricadas. Implementamos una función de multiplicación personalizada donde paralelizamos el bucle más externo[cite: 1].

Utilizando el decorador `@njit (parallel=True)` y la instrucción `prange` de Numba, logramos que cada hilo de la CPU tome un subconjunto de filas distinto y las calcule de forma independiente[cite: 1]. Esto evita dependencias de datos y nos permite usar el 100 % de la capacidad del procesador simultáneamente.

4.2. Procesamiento de Canales RGB

Para manejar imágenes a color, la metodología establece separar la imagen original en tres matrices bidimensionales (una por cada color: Rojo, Verde y Azul)[cite: 1]. Cada matriz se envía de forma independiente por el flujo del PCA (centrado, covarianza, recorte de vectores y compresión). Finalmente, las tres matrices comprimidas se restauran y se vuelven a apilar para formar la imagen a color final.

4.3. El Problema de los Formatos: ¿Por qué usamos BMP en lugar de JPG?

Una parte fundamental de nuestra metodología fue decidir cómo íbamos a medir qué tanto se redujo el peso de la imagen. Al principio consideramos usar imágenes en formato `.jpg`, pero nos topamos con un problema técnico importante.

Los formatos como el `.jpg` utilizan algoritmos de compresión con pérdida (lossy compression) que descartan información visual para reducir drásticamente el peso del archivo en el disco duro[cite: 1]. El problema es que, sin importar cuánto pese el archivo `.jpg`, cuando Python carga esa imagen a la memoria RAM para procesarla matemáticamente, la "descomprime" la despliega como una matriz densa donde cada píxel ocupa 32 bits de espacio real[cite: 1].

Si comparáramos el peso de nuestros vectores resultantes contra el peso de un archivo `.jpg`, la comparación iba a ser injusta y los números no tendrían sentido matemático. Por esta razón, establecimos en la metodología que "debíamos utilizar el formato Bitmap (`.bmp`)"[cite: 1]. El formato BMP almacena los píxeles crudos de la imagen sin ningún algoritmo de compresión superpuesto[cite: 1]. Al comparar nuestra matriz matemática comprimida contra un archivo BMP crudo, pudimos demostrar la verdadera reducción de dimensionalidad matemática que logra el PCA.

5. Diseño del Experimento y Evaluación

Para comprobar nuestra hipótesis y medir el rendimiento del código, utilizaremos hardware multinúcleo. El hiperparámetro K (los componentes a conservar) será fijado de manera manual.

Evaluaremos el sistema registrando:

- **El Tiempo Secuencial (T_1):** El tiempo total de procesamiento en un solo núcleo.
- **El Tiempo Paralelo (T_p):** El tiempo total dividiendo el trabajo con Numba.

Con estos datos calcularemos el **Speedup** ($S = T_1/T_p$), que nos dirá matemáticamente cuántas veces más rápido corre nuestra propuesta paralela en comparación a la clásica. Posteriormente, compararemos el peso original del archivo `.bmp` contra el peso del archivo binario comprimido (`.npz`) generado por nuestro programa.

6. Conclusiones Esperadas y Trabajo Futuro

Con esta metodología esperamos demostrar de manera práctica los beneficios del cómputo paralelo. Al paralelizar el núcleo matemático del PCA usando Numba, proyectamos alcanzar un Speedup evidente y comprobar que nuestro archivo `.npz` pesará solo una fracción del archivo `.bmp` original. Demostraremos que es posible construir un mecanismo efectivo de reducción de dimensionalidad a mano en un entorno concurrente.

Como limitación de este proyecto, sabemos que el cálculo de la descomposición de los autovalores se mantiene de forma secuencial[cite: 1]. Para trabajos futuros, el siguiente gran paso sería migrar toda esta arquitectura hacia la programación en GPU utilizando plataformas como CUDA, lo que permitiría procesar lotes masivos de imágenes en milisegundos.

Referencias

- [1] Shlens, J. (2014). *A tutorial on principal component analysis*. Recuperado de: https://www.researchgate.net/publication/272821354_A_Tutorial_on_Principal_Component_Analysis[cite: 1]
- [2] Lam, S. K., Pitrou, A., & Seibert, S. (2015). *Numba: A LLVM-based Python JIT compiler*. En Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (pp. 1-6). Association for Computing Machinery. <https://doi.org/10.1145/2833157.2833162>[cite: 1]
- [3] Rauber, T., & Rünger, G. (2013). *Parallel programming: For multicore and cluster systems* (2.^a ed.). Springer Science & Business Media[cite: 1].